The Complete Guide to

Git & GitHub

Commands · Concepts · Workflows · Best Practices

From Beginner to Advanced

A Comprehensive Reference for CS Students & Engineers

Version 1.0 · 2025

Chapter 1: Introduction to Version Control & Git

1.1 What Is Version Control?

Version control is a system that records changes to files over time so you can recall specific versions later. Think of it like **save states in a video game** — every time you make meaningful progress, you save a checkpoint. If something goes wrong, you can go back to any previous checkpoint without losing all your work.

Without version control, a developer might maintain copies like `project_v1.py`, `project_v2_FINAL.py`, `project_v2_FINAL_really.py`. This is error-prone, wastes space, and makes collaboration nearly impossible.

Types of Version Control Systems

- **Local VCS:** Stores version history on your local machine only. Simple but not collaborative.
- **Centralized VCS (CVCS):** A single central server stores all versioned files (e.g., SVN, CVS). Everyone checks out from that one place. Risk: if the server goes down, no one can collaborate.
- **Distributed VCS (DVCS):** Every client has a full copy (clone) of the entire repository, including full history. Git is the most popular DVCS.

1.2 What Is Git?

Git is a **free, open-source, distributed version control system** created by **Linus Torvalds** in 2005 for managing the Linux kernel source code. It is now the world's most widely used version control system.

> ✦ **Key Design Goals of Git:** Speed, simple design, strong support for non-linear development (thousands of parallel branches), fully distributed, and able to handle large projects like the Linux kernel efficiently.

How Git Thinks About Data

Most other VCS systems think of data as **a set of files and the changes made to each file over time** (delta-based storage). Git thinks about data differently — as a **stream of snapshots**.

Every time you commit, Git takes a **snapshot** of what all your files look like at that moment and stores a reference to that snapshot. If a file has not changed, Git does not store the file again — it just links to the previous identical file already stored. This makes Git more like a **mini filesystem** with powerful tools built on top.

1.3 The Three States of Git

Git has three main states that your files can reside in:

State	Meaning

`Modified`	You have changed the file but have not committed it to your database yet
`Staged`	You have marked a modified file in its current version to go into your next commit snapshot
`Committed`	The data is safely stored in your local Git database

This leads to the three main sections of a Git project:

- **Working Tree:** A single checkout of one version of the project, pulled from the Git database for you to use and modify.
- **Staging Area (Index):** A file in your Git directory that stores what will go into your next commit. Also called the 'index'.
- **Git Directory (.git):** Where Git stores the metadata and object database for your project. This is the most important part — it's what is copied when you clone.

1.4 Installing Git

Linux (Debian/Ubuntu)

Install Git on Debian/Ubuntu

```
sudo apt update
sudo apt install git
git --version
```

macOS

Install Git on macOS

```
# Option 1: Using Homebrew
brew install git

# Option 2: Xcode Command Line Tools
xcode-select --install
```

Windows

Download from `https://git-scm.com/download/win`. The installer includes Git Bash, a Unix-style terminal.

1.5 First-Time Git Configuration

Git comes with a tool called `git config` that lets you get and set configuration variables. These variables control all aspects of how Git looks and operates.

Git Global Configuration

```
# Set your identity (required before any commit)
git config --global user.name "Your Full Name"
git config --global user.email "you@example.com"

# Set default branch name to 'main'
git config --global init.defaultBranch main

# Set default editor (VS Code example)
git config --global core.editor "code --wait"

# Set line endings (Linux/Mac)
git config --global core.autocrlf input

# Set line endings (Windows)
git config --global core.autocrlf true

# Enable colored output
git config --global color.ui auto

# View all your settings
git config --list

# View a specific setting
git config user.name
```

Config Levels: System (`--system`) applies to every user on the machine. Global (`--global`) applies to your user account. Local (`--local`) applies only to the specific repository. Local overrides global, which overrides system.

Chapter 2: Core Git Operations

2.1 Initializing & Cloning Repositories

git init — Create a New Repository

The `git init` command creates a new Git repository. It can be used to convert an existing unversioned project to a Git repository or to initialize a new empty repository.

git init

```
# Initialize a new repo in the current directory
git init

# Initialize a new repo in a named directory
git init my-project

# Initialize a bare repository (for servers, no working tree)
git init --bare my-project.git
```

After running `git init`, Git creates a hidden `.git` directory inside your project. This directory contains all the metadata and object database: `HEAD`, `config`, `objects/`, `refs/`, and more. Never manually edit files inside `.git` unless you know exactly what you're doing.

git clone — Copy a Repository

The `git clone` command downloads a full copy of an existing repository, including all files, history, and branches.

git clone

```
# Clone via HTTPS
git clone https://github.com/user/repo.git

# Clone via SSH (requires SSH key setup)
git clone git@github.com:user/repo.git

# Clone into a specific directory name
git clone https://github.com/user/repo.git my-folder

# Clone only the latest snapshot (shallow clone — faster)
git clone --depth=1 https://github.com/user/repo.git

# Clone a specific branch
git clone --branch develop https://github.com/user/repo.git
```

```
# Clone with all submodules
git clone --recurse-submodules https://github.com/user/repo.git
```

2.2 Recording Changes — add, status, commit

git status — Inspect the Working Tree

One of the most frequently used commands. Shows the state of the working directory and staging area.

git status

```
# Full status output
git status

# Short/compact status
git status -s
git status --short

# Show branch info in short format
git status -sb
```

Understanding the `git status -s` output:

Symbol	Meaning
`??`	Untracked file (not yet added to Git)
`A`	New file staged for commit (added)
`M`	Modified file that is staged
` M`	Modified file that is NOT staged (in working tree only)
`MM`	Modified, staged, then modified again
`D`	Deleted file that is staged
`R`	Renamed file

git add — Stage Changes

The `git add` command adds file changes to the staging area (index), preparing them for the next commit.

git add

```
# Stage a specific file
git add filename.py

# Stage multiple files
git add file1.py file2.py
```

```
# Stage all changes in current directory (tracked + untracked)
git add .

# Stage all tracked file changes (not new files)
git add -u

# Stage ALL changes across the entire repo
git add -A

# Interactively choose which hunks (parts) of a file to stage
git add -p
git add --patch

# Stage a directory
git add src/
```

★ **git add -p (Patch Mode)** is extremely powerful. It shows you each change hunk and asks: `y` stage this hunk, `n` skip it, `s` split it into smaller hunks, `e` manually edit the hunk. This lets you make one logical commit even if your working tree has multiple unrelated changes.

git commit — Save a Snapshot

The `git commit` command saves the staged snapshot to the project history. Each commit is a permanent snapshot with a unique SHA-1 hash identifier.

git commit

```
# Commit with a message
git commit -m "feat: add user authentication module"

# Commit and open editor for a detailed message
git commit

# Stage all tracked files AND commit in one step
git commit -am "fix: correct off-by-one error in loop"

# Amend the last commit (change message or add forgot files)
git commit --amend -m "corrected commit message"

# Amend without changing the message
git commit --amend --no-edit

# Create an empty commit (useful for triggering CI)
```

```
git commit --allow-empty -m "chore: trigger pipeline"
```

Writing Good Commit Messages

A well-crafted commit message is one of the most important habits in professional development.

Conventional Commits Format

```
# Format: <type>(<scope>): <short summary>
#
# Types: feat, fix, docs, style, refactor, test, chore, perf
#
# Example of a great multi-line commit message:
feat(auth): implement JWT token refresh mechanism

Add automatic token refresh before expiry to improve UX.
The client now checks token expiry 5 minutes before it expires
and silently refreshes in the background.

Closes #142
Co-authored-by: Alice <alice@example.com>
```

2.3 Viewing History — log & diff

git log — Browse Commit History

git log

```
# Default log
git log

# One line per commit
git log --oneline

# Graph view of branches
git log --oneline --graph --all --decorate

# Show last N commits
git log -5

# Show commits by a specific author
git log --author="Alice"

# Show commits in a date range
git log --after="2024-01-01" --before="2024-06-01"
```

```
# Show commits that changed a specific file
git log -- path/to/file.py

# Show commits with diff (patch)
git log -p

# Show commits with stat (files changed summary)
git log --stat

# Search commits by message content
git log --grep="authentication"

# Search commits by code content (pickaxe)
git log -S "def login_user"

# Pretty format
git log --pretty=format:"%h - %an, %ar : %s"
```

git diff — Show Changes

git diff

```
# Show unstaged changes (working tree vs staging area)
git diff

# Show staged changes (staging area vs last commit)
git diff --staged
git diff --cached

# Show changes between two commits
git diff abc1234 def5678

# Show changes between two branches
git diff main feature/login

# Show only which files changed (no content)
git diff --name-only

# Show stat summary
git diff --stat
```

```
# Show changes to a specific file
git diff HEAD -- filename.py

# Word-by-word diff instead of line-by-line
git diff --word-diff
```

2.4 Undoing Changes

One of Git's most powerful features is the ability to undo mistakes. There are multiple approaches depending on where in the workflow you are.

git restore — Discard Working Tree Changes (Git 2.23+)

git restore

```
# Discard changes in a file (revert to last commit)
git restore filename.py

# Discard all changes in working tree
git restore .

# Unstage a file (move from staging area back to working tree)
git restore --staged filename.py

# Restore file to a specific commit's version
git restore --source=HEAD~2 filename.py
```

git reset — Move HEAD and Branch Pointer

git reset

```
# Soft reset: move HEAD back N commits, keep changes staged
git reset --soft HEAD~1

# Mixed reset (default): move HEAD, unstage changes
git reset HEAD~1
git reset --mixed HEAD~1

# Hard reset: move HEAD, discard ALL changes (DESTRUCTIVE)
git reset --hard HEAD~1

# Reset to a specific commit hash
git reset --hard abc1234

# Unstage a specific file (old syntax)
```

```
git reset HEAD filename.py
```

⚠ ⚠ **Warning:** git reset --hard permanently discards uncommitted changes. Always make sure you don't need those changes before running it. If you already pushed, reset rewrites history — coordinate with your team first.

git revert — Safe Undo (Creates New Commit)

git revert

```
# Create a new commit that undoes a specific commit
git revert abc1234

# Revert multiple commits
git revert abc1234..def5678

# Revert without auto-committing (stage the revert for review)
git revert --no-commit abc1234

# Revert the last commit
git revert HEAD
```

Unlike `git reset`, `git revert` is **safe for shared/public history** because it adds a new commit rather than rewriting existing ones. Always prefer `git revert` on commits that have already been pushed.

Chapter 3: Branching & Merging

3.1 Understanding Branches

A branch in Git is simply a **lightweight movable pointer to a specific commit**. The default branch is called `main` (or `master` in older repos). When you make commits, the branch pointer moves forward automatically.

Git's branching model is its **killer feature**. Creating a branch is nearly instantaneous (it's just creating a new pointer, not copying files), and switching between branches is equally fast. This enables workflows like:

- Developing every feature in its own isolated branch
- Maintaining long-running branches (main, develop, release)
- Experimenting with risky changes safely
- Running parallel workstreams without interference

3.2 Branch Operations

Branch Management Commands

```
# List all local branches (* = current)
git branch

# List all branches including remote-tracking
git branch -a

# List remote branches only
git branch -r

# Create a new branch
git branch feature/login

# Create and switch to a new branch (modern syntax)
git switch -c feature/login

# Create and switch (old syntax)
git checkout -b feature/login

# Switch to an existing branch
git switch main
git checkout main

# Switch to the previous branch
```

```
git switch -

# Rename a branch
git branch -m old-name new-name

# Delete a merged branch
git branch -d feature/login

# Force delete an unmerged branch
git branch -D feature/login

# Show last commit on each branch
git branch -v

# Show which branches are merged into current
git branch --merged

# Show which branches are NOT yet merged
git branch --no-merged
```

3.3 Merging

Merging is the act of integrating changes from one branch into another. Git supports several merge strategies.

Fast-Forward Merge

When the target branch has no new commits since the feature branch diverged, Git simply moves the pointer forward. No new merge commit is created.

Fast-Forward Merge

```
# Switch to main and merge feature branch
git switch main
git merge feature/login

# Force a merge commit even if fast-forward is possible
git merge --no-ff feature/login

# See what would be merged before doing it
git merge --no-commit --no-ff feature/login
```

Three-Way Merge

When both branches have diverged (both have new commits since the split), Git creates a **merge commit** with two parents. Git uses the common ancestor to perform a three-way merge.

Three-Way Merge

```
# Standard merge (creates merge commit if branches diverged)
git merge feature/payment

# If there are conflicts, after resolving them:
git add resolved-file.py
git commit

# Abort a merge in progress
git merge --abort
```

Squash Merge

Squash Merge

```
# Squash all feature commits into one before merging
git merge --squash feature/messy-branch
git commit -m 'feat: add payment processing'
```

3.4 Resolving Merge Conflicts

A conflict occurs when both branches have modified the same part of the same file. Git cannot automatically determine what the correct final content should be.

Resolving Conflicts

```
# After a failed merge, check which files conflict
git status

# The conflicted file will contain conflict markers:
<<<<<<< HEAD
This is the version from your current branch (HEAD)
=======
This is the version from the branch being merged
>>>>>>> feature/login

# Step 1: Open the file and resolve manually
# Step 2: Remove all conflict markers
# Step 3: Stage the resolved file
git add resolved-file.py

# Step 4: Complete the merge
```

```
git commit

# Use a merge tool (configure mergetool first)
git mergetool
```

3.5 Rebasing

Rebase is an alternative to merging. Instead of creating a merge commit, rebase **replays your commits on top of another branch**, resulting in a linear history.

git rebase

```
# Rebase current branch onto main
git rebase main

# Interactive rebase — rewrite last N commits
git rebase -i HEAD~4

# Interactive rebase options:
#  pick   = use commit as-is
#  reword = use commit, but edit the message
#  edit   = use commit, but stop to amend
#  squash = combine with previous commit
#  fixup  = like squash but discard this message
#  drop   = remove the commit entirely

# Continue after resolving conflicts during rebase
git rebase --continue

# Skip a commit during rebase
git rebase --skip

# Abort rebase and return to original state
git rebase --abort

# Rebase onto another branch (move feature onto main)
git rebase --onto main feature-base feature-branch
```

⚠ **Golden Rule of Rebasing:** Never rebase commits that exist outside your local repository and that other people may have based work on. Rebasing rewrites commit history (new SHA-1 hashes). If you push rebased commits to a shared branch, your teammates' histories will conflict.

3.6 Cherry-Picking

Cherry-picking applies the changes introduced by specific commits onto your current branch — without merging the entire branch.

git cherry-pick

```
# Apply a specific commit to current branch
git cherry-pick abc1234

# Cherry-pick multiple commits
git cherry-pick abc1234 def5678

# Cherry-pick a range of commits
git cherry-pick abc1234^..def5678

# Cherry-pick without auto-committing
git cherry-pick --no-commit abc1234

# Continue after conflict resolution
git cherry-pick --continue

# Abort
git cherry-pick --abort
```

Chapter 4: Remote Repositories & GitHub

4.1 Understanding Remotes

Remote repositories are versions of your project hosted on the internet or network. The most common remote is **origin**, which is the name Git gives the server you cloned from.

Remote Management

```
# View remotes (name + URL)
git remote -v

# Add a new remote
git remote add origin https://github.com/user/repo.git

# Add a second remote (e.g., upstream for forks)
git remote add upstream https://github.com/original/repo.git

# Rename a remote
git remote rename origin old-origin

# Remove a remote
git remote remove upstream

# Change a remote's URL (e.g., switch HTTPS to SSH)
git remote set-url origin git@github.com:user/repo.git

# Inspect a remote (shows tracked branches, etc.)
git remote show origin
```

4.2 Pushing & Pulling

git fetch — Download Without Merging

Fetch downloads remote commits, files, and references into your local repo **without modifying your working tree**. It updates your remote-tracking branches.

git fetch

```
# Fetch from the default remote (origin)
git fetch

# Fetch from a specific remote
git fetch upstream
```

```
# Fetch all remotes
git fetch --all

# Fetch and prune deleted remote branches
git fetch --prune
git fetch -p

# Fetch a specific branch
git fetch origin main
```

git pull — Fetch + Merge

Pull is essentially `git fetch` followed by `git merge`. It downloads changes and immediately integrates them into your current branch.

git pull

```
# Pull and merge
git pull

# Pull and rebase instead of merge (keeps history linear)
git pull --rebase
git pull -r

# Pull from a specific remote and branch
git pull origin main

# Pull all remote branches
git pull --all
```

git push — Upload to Remote

git push

```
# Push current branch to its tracked remote branch
git push

# Push and set upstream tracking for first push
git push -u origin feature/login
git push --set-upstream origin feature/login

# Push a specific local branch to remote
git push origin feature/login
```

```
# Push all branches
git push --all origin

# Push tags
git push --tags

# Delete a remote branch
git push origin --delete feature/old-branch

# Force push (DESTRUCTIVE — rewrites remote history)
git push --force
git push -f

# Safer force push (only if remote hasn't changed)
git push --force-with-lease
```

✦ **--force-with-lease vs --force:** Always prefer `--force-with-lease` over `--force` when you must force push. It refuses to overwrite remote if someone else has pushed since your last fetch, preventing accidental data loss.

4.3 GitHub — The Platform

GitHub is a web-based hosting service for Git repositories with collaboration features built on top: pull requests, issues, Actions (CI/CD), Pages, Packages, and more.

Setting Up SSH Keys for GitHub

SSH Key Setup for GitHub

```
# Step 1: Generate an Ed25519 SSH key pair
ssh-keygen -t ed25519 -C "you@example.com"

# Step 2: Start the ssh-agent
eval $(ssh-agent -s)

# Step 3: Add your key to the agent
ssh-add ~/.ssh/id_ed25519

# Step 4: Copy your public key
cat ~/.ssh/id_ed25519.pub
# Paste this into GitHub → Settings → SSH Keys

# Step 5: Test connection
ssh -T git@github.com
```

```
# Should see: Hi username! You've successfully authenticated.
```

4.4 Pull Requests (PRs)

A Pull Request is a GitHub mechanism to **propose changes** from one branch to another. It enables code review, discussion, and CI/CD checks before merging.

PR Workflow

- **Fork or Branch:** Either fork the repo (for open source) or create a feature branch (for team repos)
- **Make Changes:** Commit your work on the feature branch
- **Push:** Push the branch to GitHub
- **Open PR:** GitHub UI → Compare & pull request → fill title, description, assignees, labels
- **Review:** Team reviews, requests changes, or approves
- **Merge:** Once approved and CI passes, merge via GitHub UI
- **Clean up:** Delete the feature branch after merge

GitHub CLI for PRs

GitHub CLI — Pull Requests

```
# Install GitHub CLI
# https://cli.github.com

# Create a PR from current branch
gh pr create --title "feat: add login page" --body "Closes #23"

# List open PRs
gh pr list

# View a PR
gh pr view 42

# Check out a PR locally for review
gh pr checkout 42

# Merge a PR
gh pr merge 42 --squash

# Review a PR
gh pr review 42 --approve
gh pr review 42 --request-changes --body "Please fix the tests"
```

4.5 GitHub Issues & Projects

GitHub CLI — Issues

```
# Create an issue
gh issue create --title "Bug: login fails on Safari" --body "Steps to
reproduce..."

# List issues
gh issue list

# Close an issue
gh issue close 15

# Link commit to issue (auto-close on merge)
# In commit message:
git commit -m "fix: resolve Safari CORS issue  Closes #15"
```

Chapter 5: Advanced Git Techniques

5.1 git stash — Save Work Temporarily

Stash temporarily shelves (stashes) changes so you can switch context (e.g., to fix an urgent bug), then come back and re-apply them.

git stash

```
# Stash current changes
git stash
git stash push

# Stash with a descriptive message
git stash push -m "WIP: refactoring auth module"

# Also stash untracked files
git stash push -u
git stash push --include-untracked

# Also stash ignored files
git stash push -a

# Stash specific files only
git stash push -- src/auth.py

# List all stashes
git stash list

# Apply most recent stash (keep it in stash list)
git stash apply

# Apply a specific stash
git stash apply stash@{2}

# Apply AND remove most recent stash
git stash pop

# Show what's in a stash
git stash show -p stash@{0}

# Create a branch from a stash
git stash branch new-feature stash@{0}
```

```
# Drop (delete) a specific stash
git stash drop stash@{1}

# Clear ALL stashes
git stash clear
```

5.2 Tagging

Tags mark specific points in history as important — typically used for **release versions**.

git tag

```
# Create a lightweight tag (just a pointer)
git tag v1.0.0

# Create an annotated tag (with message, tagger info, date)
git tag -a v1.0.0 -m "Release version 1.0.0"

# Tag a specific past commit
git tag -a v0.9.0 abc1234 -m "Beta release"

# List all tags
git tag
git tag -l

# Filter tags
git tag -l 'v1.*'

# Show tag details
git show v1.0.0

# Push a single tag to remote
git push origin v1.0.0

# Push all tags
git push origin --tags

# Delete a local tag
git tag -d v1.0.0

# Delete a remote tag
git push origin --delete v1.0.0
```

```
git push origin :refs/tags/v1.0.0

# Check out code at a tag (detached HEAD)
git checkout v1.0.0
```

5.3 git reflog — Your Safety Net

The reflog records every time HEAD moves — even after resets, rebases, and branch deletions. It's your **last resort recovery tool**.

git reflog

```
# View reflog
git reflog

# View reflog for a specific branch
git reflog show feature/login

# Recover a commit after accidental reset
git reflog
# Find the SHA before the bad reset, e.g. abc1234
git reset --hard abc1234

# Recover a deleted branch
git reflog
# Find last commit on that branch
git checkout -b recovered-branch abc1234

# Expire old reflog entries (cleanup)
git reflog expire --expire=90.days refs/heads/main
```

5.4 git bisect — Binary Search for Bugs

Bisect uses binary search to find which commit introduced a bug. Given a good commit and a bad commit, Git automatically checks out the midpoint and asks you to test.

git bisect

```
# Start bisect session
git bisect start

# Mark current commit as bad
git bisect bad
```

```
# Mark a known good commit
git bisect good v1.0.0
git bisect good abc1234

# Git checks out midpoint — test your code then tell Git:
git bisect good    # if this commit is fine
git bisect bad     # if this commit has the bug

# Git will narrow down and eventually identify the culprit

# Automated bisect with a test script
git bisect run python test.py
# (script exits 0 for good, non-zero for bad)

# End bisect session (returns to original HEAD)
git bisect reset
```

5.5 Submodules

Submodules allow you to keep a Git repository as a subdirectory of another Git repository — useful for including libraries or shared components.

git submodule

```
# Add a submodule
git submodule add https://github.com/user/lib.git libs/lib

# Initialize submodules after cloning
git submodule init
git submodule update

# Clone with all submodules in one step
git clone --recurse-submodules https://github.com/user/repo.git

# Update all submodules to their latest remote commit
git submodule update --remote

# Pull changes including submodule updates
git pull --recurse-submodules

# Run a command in each submodule
git submodule foreach git pull origin main
```

```
# Remove a submodule
git submodule deinit libs/lib
git rm libs/lib
```

5.6 git worktree — Multiple Working Trees

Worktrees let you check out multiple branches of a repo into different directories simultaneously. Useful for working on a hotfix while keeping your main work untouched.

git worktree

```
# Add a new worktree for a branch
git worktree add ../hotfix-tree hotfix/critical-bug

# Create a worktree with a new branch
git worktree add -b hotfix/new-fix ../hotfix-tree main

# List all worktrees
git worktree list

# Remove a worktree
git worktree remove ../hotfix-tree

# Prune stale worktree info
git worktree prune
```

Chapter 6: Git Workflows & Team Best Practices

6.1 Git Flow

Git Flow is a branching model with clearly defined roles for different branches and how they should interact.

Branch Structure

Branch	Purpose
`main / master`	Production-ready code only. Never commit directly to this.
`develop`	Integration branch. Features merge here first.
`feature/*`	One branch per feature, branched from develop.
`release/*`	Preparation for a production release. Bug fixes only.
`hotfix/*`	Emergency fixes branched directly from main.

Git Flow Commands

```
# Start a feature
git switch develop
git switch -c feature/user-profile
# ... work, commit ...
git switch develop
git merge --no-ff feature/user-profile
git branch -d feature/user-profile

# Start a release
git switch -c release/1.2.0 develop
# ... bump version, fix bugs only ...
git switch main && git merge --no-ff release/1.2.0
git tag -a v1.2.0 -m 'Release 1.2.0'
git switch develop && git merge --no-ff release/1.2.0
git branch -d release/1.2.0

# Start a hotfix
git switch -c hotfix/1.2.1 main
# ... fix bug ...
git switch main && git merge --no-ff hotfix/1.2.1
git tag -a v1.2.1
git switch develop && git merge --no-ff hotfix/1.2.1
git branch -d hotfix/1.2.1
```

6.2 GitHub Flow (Simpler)

GitHub Flow is a simplified workflow for teams doing continuous deployment. There is only one long-lived branch: `main` .

- Anything in main is deployable
- Create descriptive branches off main for new work
- Push to GitHub and open a Pull Request early
- After review and CI pass, merge into main
- Deploy immediately after merging

GitHub Flow

```
git switch main && git pull
git switch -c feature/dark-mode
# ... commits ...
git push -u origin feature/dark-mode
# Open PR on GitHub
# Get reviews, iterate
# Merge PR
git switch main && git pull
git branch -d feature/dark-mode
```

6.3 Trunk-Based Development

Trunk-Based Development (TBD) is a high-velocity approach where all developers commit directly to a single shared trunk (`main`), using feature flags to hide incomplete features from users.

- Very short-lived branches (hours, not days)
- Feature flags/toggles to safely ship incomplete code
- Heavy reliance on automated testing and CI/CD
- Favored by Google, Meta, Netflix for large codebases

6.4 Commit Best Practices

Practice	Detail
`Atomic commits`	One commit = one logical change. Don't mix unrelated changes.
`Present tense`	"Add feature" not "Added feature"
`Conventional Commits`	Use prefixes: feat, fix, docs, refactor, test, chore
`Reference issues`	Include "Closes #42" or "Refs #99" in body
`50/72 rule`	Subject ≤ 50 chars; body lines wrapped at 72 chars

`Don't commit secrets`	Use .gitignore and environment variables for credentials

6.5 .gitignore — Excluding Files

The `.gitignore` file specifies patterns of files that Git should ignore and not track.

.gitignore patterns

```
# Ignore a specific file
secrets.env

# Ignore all .log files
*.log

# Ignore a directory
node_modules/
__pycache__/
.venv/

# Ignore everything in a directory but keep the dir
logs/*
!logs/.gitkeep

# Negation — track this despite parent rule
!important.log

# Ignore OS files
.DS_Store
Thumbs.db

# Ignore IDE files
.vscode/
.idea/
*.swp

# Ignore build output
dist/
build/
*.pyc
*.class
```

Global .gitignore

```
# Create a global gitignore (applies to ALL your repos)
```

```
git config --global core.excludesfile ~/.gitignore_global

# Check if a file is ignored
git check-ignore -v filename.log

# Remove a file from tracking without deleting it
git rm --cached filename.env

# Templates at: https://github.com/github/gitignore
```

Chapter 7: GitHub Actions & CI/CD

7.1 What Are GitHub Actions?

GitHub Actions is a CI/CD platform built into GitHub. It lets you automate workflows — running tests, building Docker images, deploying to cloud — triggered by GitHub events like pushes, PRs, or schedules.

Core Concepts

Concept	Description
Workflow	An automated process defined in a YAML file in .github/workflows/
Event	A trigger: push, pull_request, schedule, workflow_dispatch, etc.
Job	A set of steps that run on the same runner machine
Step	An individual task: run a command or use an Action
Action	A reusable unit of code (from Marketplace or custom)
Runner	The virtual machine that executes the job (ubuntu-latest, windows, mac)

7.2 Basic Workflow — Python CI

Python CI Workflow

```
# .github/workflows/ci.yml
name: Python CI

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        python-version: ['3.10', '3.11', '3.12']

    steps:
```

```
      - uses: actions/checkout@v4

      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest pytest-cov ruff

      - name: Lint with ruff
        run: ruff check .

      - name: Run tests with coverage
        run: |
          pytest --cov=src --cov-report=xml

      - name: Upload coverage to Codecov
        uses: codecov/codecov-action@v4
        with:
          file: ./coverage.xml
```

7.3 Deploy Workflow — Docker + AWS

Docker Deployment Workflow

```
# .github/workflows/deploy.yml
name: Build and Deploy

on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production

    steps:
      - uses: actions/checkout@v4
```

```
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: us-east-1

      - name: Login to ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build, tag, and push Docker image
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
          IMAGE_TAG: ${{ github.sha }}
        run: |
          docker build -t $ECR_REGISTRY/my-app:$IMAGE_TAG .
          docker push $ECR_REGISTRY/my-app:$IMAGE_TAG

      - name: Deploy to ECS
        run: |
          aws ecs update-service \
            --cluster production \
            --service my-app \
            --force-new-deployment
```

7.4 Secrets & Environment Variables

Secrets & Variables

```
# In your workflow, reference secrets like this:
env:
  API_KEY: ${{ secrets.MY_API_KEY }}
  DATABASE_URL: ${{ secrets.DATABASE_URL }}

# Repository-level: Settings → Secrets → Actions
# Organization-level: shared across repos
# Environment-level: require deployment approval

# Default GitHub variables (always available):
${{ github.sha }}         # commit hash
${{ github.ref }}         # branch or tag ref
```

```
${{ github.actor }}        # username who triggered
${{ github.repository }}   # owner/repo-name
${{ github.run_number }}   # incremental build number
```

Chapter 8: GitHub Security, Pages & Advanced Features

8.1 Branch Protection Rules

Branch protection rules enforce workflows and prevent mistakes on important branches like `main`.

Rule	What It Enforces
`Require PR reviews`	Minimum N approvals before merging
`Dismiss stale reviews`	Re-review required if new commits are pushed
`Require status checks`	CI must pass before merge
`Require signed commits`	All commits must be GPG-signed
`Restrict pushes`	Only specific users/teams can push directly
`Require linear history`	No merge commits — must squash or rebase

8.2 GPG Signing Commits

GPG Commit Signing

```
# Generate a GPG key
gpg --full-generate-key
# Choose RSA 4096, expiry, your GitHub email

# List your GPG keys
gpg --list-secret-keys --keyid-format=long

# Copy the key ID (after 'rsa4096/')
git config --global user.signingkey YOUR_KEY_ID

# Sign commits by default
git config --global commit.gpgsign true

# Sign a single commit manually
git commit -S -m 'signed commit message'

# Export public key to add to GitHub
gpg --armor --export YOUR_KEY_ID
# Paste in GitHub → Settings → SSH and GPG Keys
```

8.3 GitHub Pages

GitHub Pages publishes static websites directly from a repository. Free for public repos, supports custom domains and HTTPS.

GitHub Pages

```
# Option 1: Publish from /docs folder on main
# Settings → Pages → Source: main branch, /docs folder

# Option 2: gh-pages branch
git switch --orphan gh-pages
echo '<h1>My Site</h1>' > index.html
git add . && git commit -m 'Initial pages commit'
git push origin gh-pages

# Option 3: GitHub Actions workflow (most flexible)
# .github/workflows/pages.yml — build then deploy

# Custom domain
echo 'www.yourdomain.com' > CNAME
git add CNAME && git commit -m 'Add custom domain'
git push
```

8.4 Security Best Practices

- **Never commit secrets:** Use .gitignore, environment variables, or secret managers (HashiCorp Vault, AWS Secrets Manager)
- **Use Dependabot:** Enable in Settings → Security. Auto-creates PRs to update vulnerable dependencies
- **Enable Code Scanning:** GitHub's CodeQL or third-party tools scan for vulnerabilities in your code
- **Secret Scanning:** GitHub alerts you if it detects API keys, tokens, or passwords in your commits
- **Audit Log:** Organization admins can review every action in the org (Settings → Audit Log)
- **2FA:** Enable two-factor authentication on your GitHub account

Removing Secrets from History

```
# Scan git history for secrets (open source tool)
pip install truffleHog
trufflehog git file://.

# Or use gitleaks
gitleaks detect --source . -v
```

```
# BFG Repo-Cleaner: remove secrets from history
# (nuclear option — rewrites all history)
java -jar bfg.jar --replace-text passwords.txt repo.git
git reflog expire --expire=now --all
git gc --prune=now --aggressive
```

8.5 Useful Git Aliases

Git Aliases

```
# Add aliases to ~/.gitconfig or via git config

git config --global alias.st "status -sb"
git config --global alias.lg "log --oneline --graph --all --decorate"
git config --global alias.undo "reset --soft HEAD~1"
git config --global alias.staged "diff --staged"
git config --global alias.last "log -1 HEAD --stat"
git config --global alias.aliases "config --get-regexp alias"
git config --global alias.unstage "restore --staged"
git config --global alias.sync "!git fetch --all --prune && git pull"

# Usage
git st
git lg
git undo
```

8.6 Quick Reference — Most Used Commands

Command	Description
`git init`	Initialize a new repository
`git clone <url>`	Clone a remote repository
`git status`	Show working tree status
`git add .`	Stage all changes
`git commit -m 'msg'`	Commit staged changes
`git push`	Push to remote
`git pull`	Fetch and merge remote changes
`git branch -a`	List all branches
`git switch -c <name>`	Create and switch to new branch
`git merge <branch>`	Merge branch into current
`git rebase main`	Rebase current branch onto main

`git stash`	Stash current changes
`git log --oneline --graph`	Visual branch history
`git diff --staged`	Show staged changes
`git reset --soft HEAD~1`	Undo last commit, keep changes staged
`git revert HEAD`	Safely undo last commit (new commit)
`git cherry-pick <hash>`	Apply a specific commit
`git bisect start`	Start binary search for bug
`git tag -a v1.0 -m 'msg'`	Create annotated tag
`git reflog`	Show all HEAD movements

— End of Guide —

Happy Coding!